

PRD — Property Management & Lease Analytics Platform

Project Type: Portfolio + Interview Showcase Project

Target Role: Yardi Software India — Software Engineer Trainee

Primary Positioning: **Software Engineer with strong SQL + Application Support + Reporting capability**

This PRD intentionally combines your two original ideas into **one flagship project**. That is the right choice for the Yardi drive because the posted role specifically calls for SQL application support, custom solution development, SSRS/Crystal Report Writer, complex SQL reporting, performance optimization, and unit testing.

1. Product Overview

1.1 Product Name

PropLedger — Property Management & Analytics Platform

A near-enterprise property-management platform that models residential and commercial real estate operations while using **SQL Server as the technical centerpiece**.

The system manages:

- Properties
- Buildings
- Units
- Owners
- Tenants
- Leases
- Rent
- Payments
- Security deposits
- Late fees
- Delinquencies
- Collections

- Maintenance
- Vendors
- Employees
- Invoices
- Expenses
- Property financials

It also provides a dedicated **reporting and analytics layer using SSRS and Crystal Reports**.

2. Why This Project Exists

The Yardi placement notice identifies the position as **Software Engineer Trainee** and explicitly lists SQL application support, custom solutions, reporting tools, complex SQL reporting, performance measurement/optimization, and unit testing among the responsibilities.

Therefore, this project should demonstrate five things exceptionally well:

Capabilit y	Project Demonstr ation
SQL	Advanced relational database + complex queries
Applicatio n Support	Real workflows, validation, error handling, troublesho oting

Reporting	SSRS + Crystal Reports
Performance	Execution plans, indexing, benchmark comparisons
Development	React application + API + database + testing

The project **should not be marketed primarily as a React project.**

React is the interface.

SQL + reporting + business logic are the core.

3. Product Vision

Build a realistic property-management system that demonstrates how a software engineer can transform complex business requirements into reliable SQL-backed applications, operational reports, optimized queries, and tested custom solutions.

The project should feel like a **small enterprise software product**, not a college CRUD assignment.

4. Target Users

The system supports all requested user groups.

Administrator

Complete system control.

Property Manager

Manages properties, units, leases, tenants and operational performance.

Leasing / Front Office Staff

Handles tenants, applications, leases and renewals.

Accountant / Finance Staff

Handles rent, payments, invoices, expenses, balances and financial reporting.

Maintenance Staff

Handles maintenance requests, work orders and vendors.

Property Owner

Views property-level financial and operational reports.

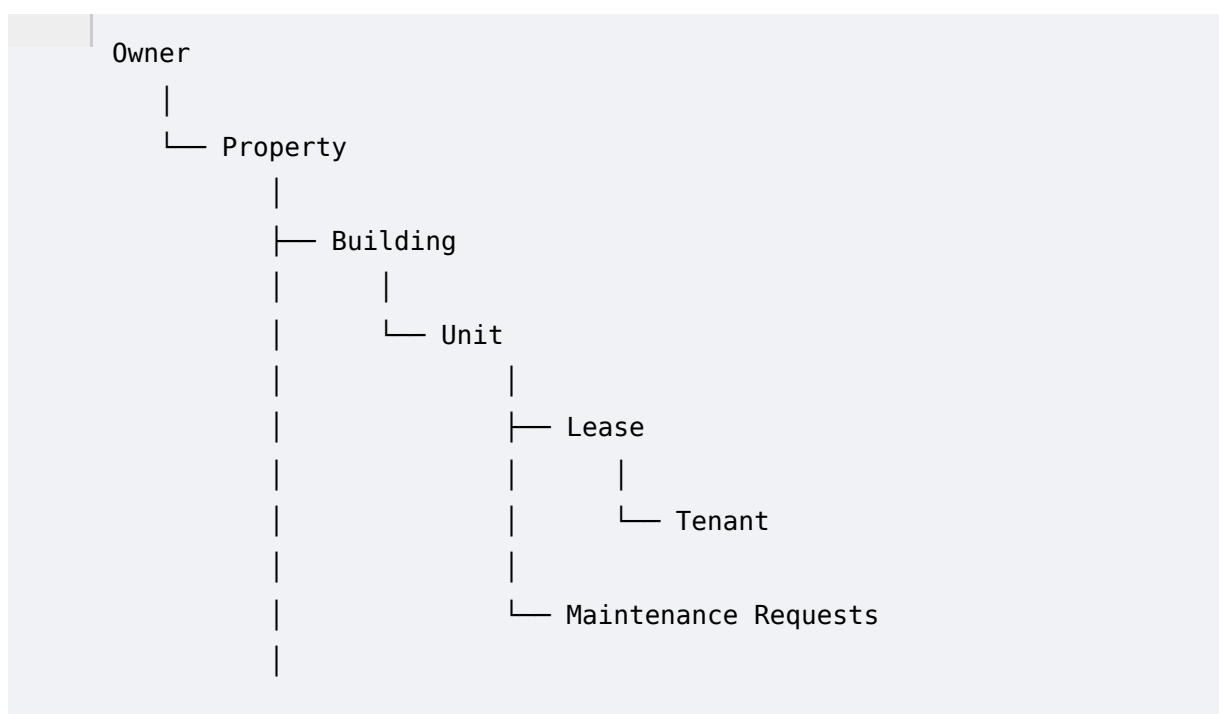
Tenant

Views lease information, rent obligations, payments and maintenance requests.

5. Domain Model

The project uses a **mixed residential + commercial** property model.

Core hierarchy



└─ Expenses
└─ Vendors

Financial flow:

Lease
↓
Rent Charge
↓
Payment
↓
Outstanding Balance
↓
Late Fee
↓
Delinquency
↓
Collection

This complete lifecycle is a critical business workflow for the project.

6. Product Goals

Primary Goals

1. Demonstrate advanced SQL proficiency.
2. Demonstrate production-style relational database design.
3. Demonstrate SSRS reporting.
4. Demonstrate Crystal Reports.
5. Demonstrate query performance analysis.
6. Demonstrate transaction management.
7. Demonstrate business-rule implementation.
8. Demonstrate role-based application design.
9. Demonstrate testing.
10. Provide a strong interview discussion project.

Secondary Goals

- Build a polished React interface.
- Provide exportable operational reports.
- Simulate realistic business data.

- Create documentation that allows another developer to understand the system.
-

7. Non-Goals

The project should **not** attempt to become a full commercial Yardi replacement.

Do not spend excessive time on:

- Microservices
- Kubernetes
- Distributed systems
- AI features
- Blockchain
- Complex DevOps infrastructure
- Mobile applications
- Elaborate animations
- Dozens of unnecessary CRUD screens

These do not materially strengthen the technical story for this particular role.

8. Proposed Technology Stack

Frontend

React.js

Recommended:

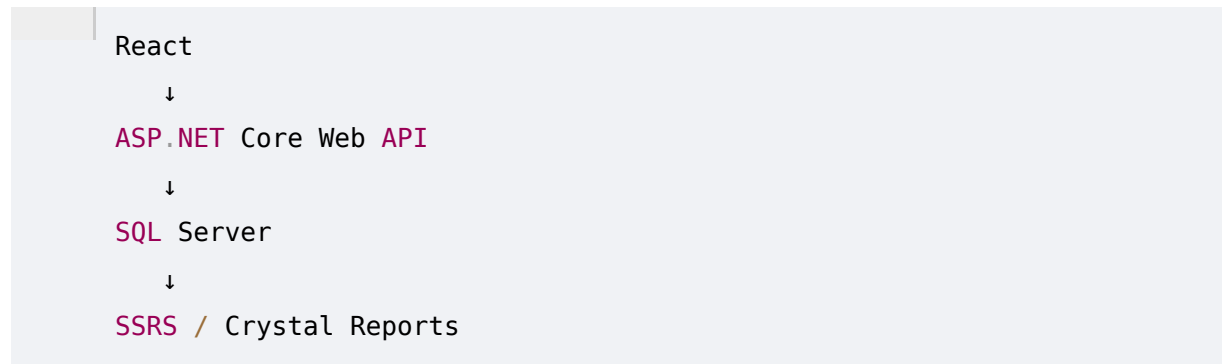
```
React
TypeScript
React Router
TanStack Query
Chart library
CSS / Tailwind or equivalent
```

The frontend should consume APIs and visualize operational data.

Backend

ASP.NET Core Web API

Recommended because it gives the project a clean enterprise-style application layer between React and SQL Server.



Database

Microsoft SQL Server

Target:

- SQL Server Developer Edition for development
- Compatible SQL design wherever practical with SQL Server Express

Database is the **central technical artifact** of the project.

Reporting

Primary

SQL Server Reporting Services (SSRS)

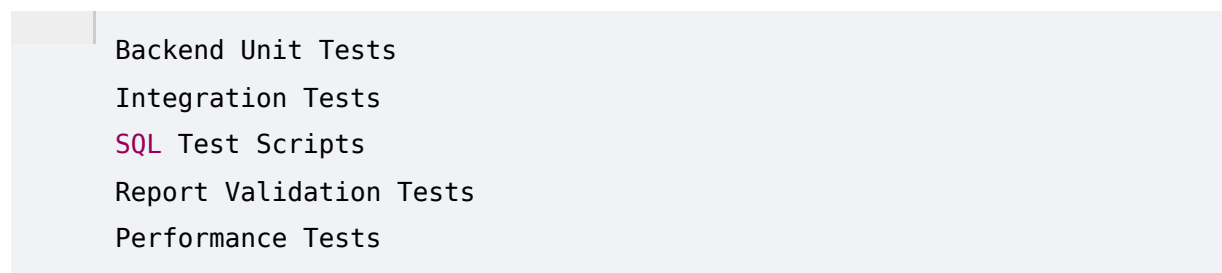
Secondary

Crystal Reports

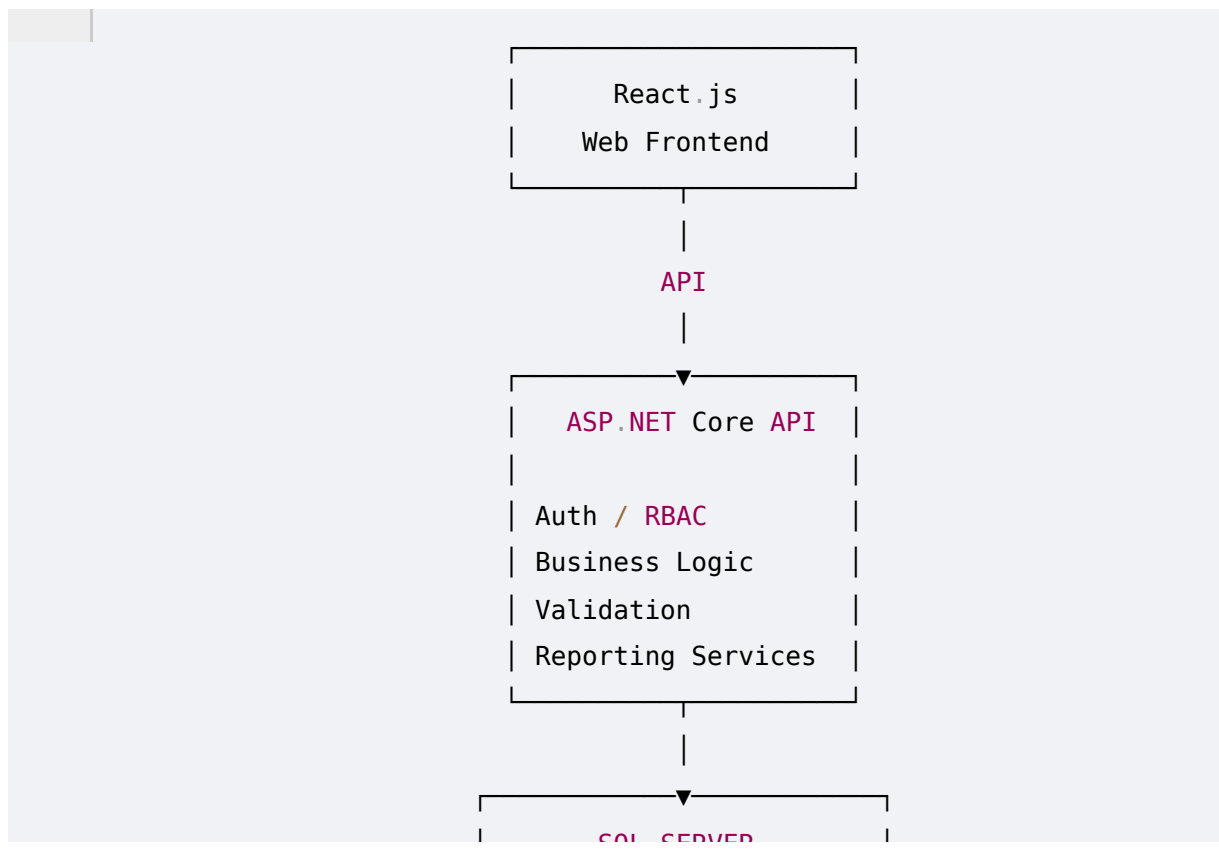
Both should use SQL Server-backed datasets.

This directly reflects the reporting tools named in the Yardi posting.

Testing



9. High-Level Architecture

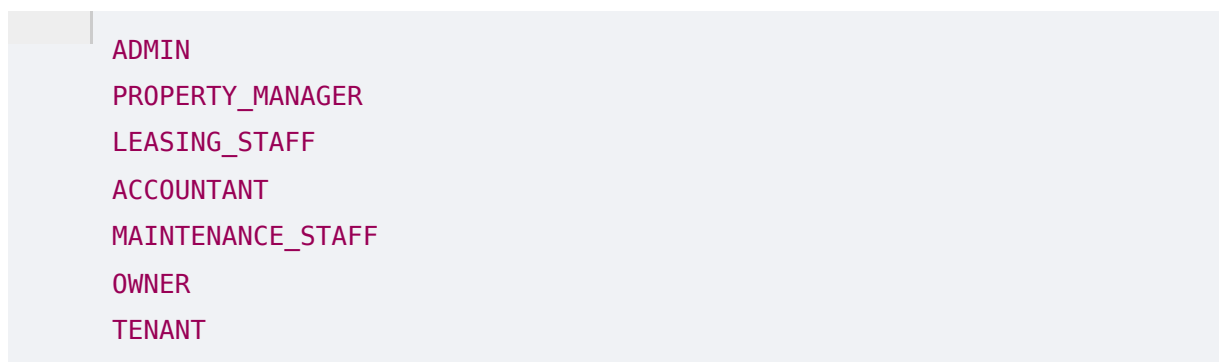


10. Functional Modules

Module 1 — Authentication & RBAC

Users log in according to their assigned role.

Roles



Permissions must be enforced server-side.

Example:

Function	Admin	Manager	Accountant	Tenant

Create property	✓	✓		
Create lease	✓	✓		
Record payment	✓		✓	
View financial reports	✓	✓	✓	limited
Create maintenance request	✓	✓		✓

11. Property Management

Users can:

- Create property
- Edit property
- Assign owner
- Define property type
- Add buildings
- Add units
- Define unit type
- Define rent
- Track occupancy
- Mark units available / occupied / maintenance

Property Types

Residential

Commercial

Mixed

Unit Status

Available
Occupied
Reserved
Maintenance
Inactive

12. Tenant Management

Each tenant has:

- Personal information
- Contact information
- Identification reference
- Occupancy details
- Lease history
- Payment history
- Outstanding balance
- Maintenance history

System should maintain historical relationships rather than overwrite important historical data.

13. Lease Management

Each lease contains:

Lease ID
Property
Unit
Tenant
Start Date
End Date
Monthly Rent
Security Deposit
Late Fee Policy
Rent Due Day
Renewal Status
Lease Status

Lease States

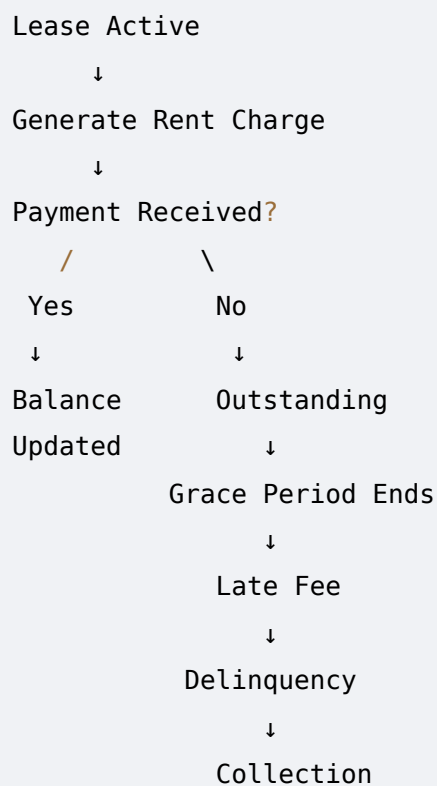
Draft
Active
Expiring
Expired
Terminated
Renewed

The system should identify upcoming expirations automatically.

14. Rent & Payment Engine

This should be one of the most technically important modules.

Monthly lifecycle



Partial Payment Example

Suppose:

Monthly Rent = ₹25,000
Payment = ₹15,000

System:

Rent Due	₹25,000
Payment	₹15,000
Outstanding	₹10,000

After applicable late-fee rules:

Outstanding = ₹10,000
Late Fee = ₹500
Total Due = ₹10,500

This logic should be implemented through properly designed SQL/business rules rather than hardcoded UI calculations.

15. Collections Module

Track:

- Outstanding rent
- Aging
- Number of overdue days
- Late fees
- Collection status
- Follow-up activity
- Settlement
- Write-off

Aging Categories

Current
1–30 Days
31–60 Days
61–90 Days
90+ Days

This gives the reporting engine useful analytical dimensions.

16. Maintenance Module

Users can create:

Maintenance Request

with:

- Unit
- Tenant
- Property
- Category
- Priority
- Description
- Created date
- Assigned employee/vendor
- Status
- Completion date
- Cost

Status

Open
Assigned
In Progress
On Hold
Resolved
Closed

The system can calculate:

Average Resolution Time
Requests per Property
Requests by Category
Maintenance Cost
Vendor Performance

17. Vendor Management

Track:

- Vendor
- Service category
- Contact information

- Assigned work
 - Cost
 - Performance
 - Completion times
-

18. Finance Module

Track:

Income

- Rent
- Late fees
- Other property income

Expenses

- Maintenance
- Utilities
- Vendor payments
- Administrative expenses
- Other operating expenses

This enables property-level profitability reports.

19. Advanced SQL Requirements

This is the heart of the project.

The database should intentionally demonstrate:

Joins

```
INNER JOIN  
LEFT JOIN  
SELF JOIN
```

Subqueries

- Scalar
- Correlated
- EXISTS
- NOT EXISTS

CTEs

Including:

- ordinary CTE
- recursive CTE

Window Functions

```
ROW_NUMBER()  
RANK()  
DENSE_RANK()  
LAG()  
LEAD()  
SUM() OVER()  
AVG() OVER()
```

PIVOT

For example:

```
Property × Monthly Rent Collection
```

Conditional aggregation

```
SUM(CASE WHEN ... THEN ... END)
```

Views

Examples:

```
vw_PropertyOccupancy  
vw_TenantOutstandingBalance  
vw_ActiveLeases  
vw_PropertyFinancialSummary
```

Stored Procedures

Examples:

```
usp_GenerateMonthlyRent  
usp_RecordPayment  
usp_GetTenantPaymentHistory  
usp_GetPropertyOccupancy  
usp_GetDelinquencyReport
```

```
usp_GetLeaseExpiryReport  
usp_GetPropertyFinancialSummary
```

Functions

Examples:

```
fn_CalculateLateFee()  
fn_GetOutstandingBalance()  
fn_GetLeaseStatus()
```

Triggers

Use selectively.

Examples:

```
Payment inserted  
→ audit record created  
  
Lease status changed  
→ status history created
```

Triggers should have clear justification; don't use triggers simply to demonstrate that you know triggers.

20. Transaction Requirements

Payment processing must be transactional.

Conceptually:

```
BEGIN TRANSACTION  
  
1. Validate lease  
2. Validate payment  
3. Insert payment  
4. Update outstanding balance  
5. Update lease/account status  
6. Record audit event  
  
COMMIT
```

On failure:

ROLLBACK

You should be able to explain in an interview why a payment workflow must be atomic.

21. Database Auditing

Maintain audit information such as:

```
CreatedBy  
CreatedAt  
ModifiedBy  
ModifiedAt
```

For sensitive entities, create history tables where appropriate.

Example:

```
LeaseHistory  
PaymentAudit  
RentAdjustmentHistory  
StatusChangeHistory
```

22. Synthetic Data Strategy

Use **SQL seed scripts + large synthetic datasets**.

The target is not merely:

```
10 properties  
20 tenants
```

Instead, build enough data for performance experiments.

Suggested initial dataset:

```
500–1,000 properties  
Several thousand units  
Several thousand tenants  
Several thousand leases  
Hundreds of thousands of rent/payment records  
Thousands of maintenance records
```

Exact numbers should depend on your machine.

The important principle is:

Large enough to make performance differences measurable.

23. Reporting System

The project should contain a dedicated **Reports Center**.

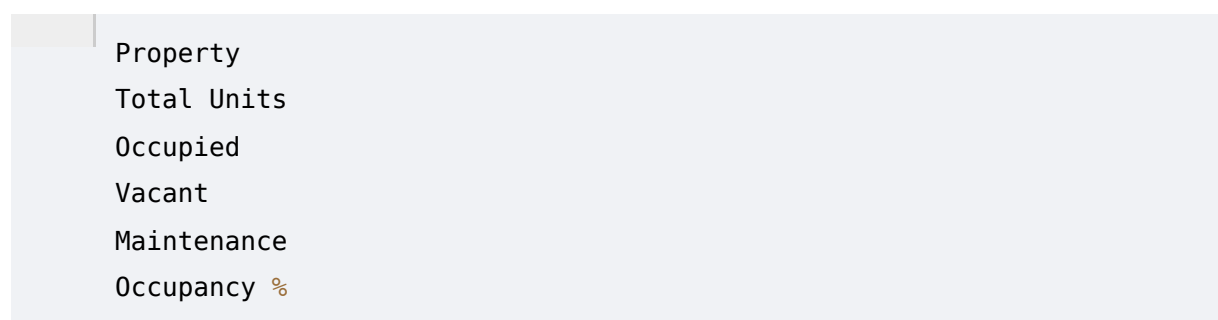
Example:



24. Required Reports

You selected **all** reports, so the PRD treats them as first-class deliverables.

1. Property Occupancy Report



2. Unit Availability Report

Filter by:

Property
Building
Unit Type
Availability
Rent Range

3. Tenant Directory

Tenant
Unit
Property
Lease
Contact
Lease Status

4. Lease Expiration Report

Parameters:

From Date
To Date
Property

Output:

Tenant
Unit
Lease End
Days Remaining
Renewal Status

5. Monthly Rent Collection Report

Expected Rent
Collected Rent
Outstanding
Late Fees
Collection %

6. Delinquency Report

Tenant
Property
Amount Due
Days Overdue
Late Fee
Aging Category
Collection Status

7. Tenant Payment History

Tenant
Lease
Invoice
Payment Date
Payment Amount
Method
Remaining Balance

8. Security Deposit Report

Track:

Tenant
Deposit
Deposit Date
Current Status
Refund Amount
Deductions

9. Maintenance Performance Report

Property
Requests
Open
Closed
Average Resolution Time
Total Cost

10. Property Income & Expense Report

Property
Income
Expenses
Net Operating Result

11. Monthly Property Summary

A management-oriented combined report.

12. Owner / Executive Dashboard Report

KPIs:

Occupancy
Revenue
Outstanding Rent
Collection %
Maintenance Cost
Lease Expirations

13. Rent Roll

A highly useful property-management report:

Property
Unit
Tenant
Lease
Monthly Rent
Status
Balance

14. Lease Renewal Report

Identify:

Expiring leases
Renewed leases
Pending renewals
Non-renewals

25. SSRS Requirements

Every major SSRS report should demonstrate some combination of:

- Parameters
- Multiple datasets where justified
- Grouping
- Sorting
- Expressions
- Conditional formatting
- Aggregations
- Drill-down
- Subreports where justified
- Page headers/footers
- Report metadata
- Export to PDF
- Export to Excel

The objective is to demonstrate that you understand the reporting workflow rather than merely generating a table.

26. Crystal Reports Requirements

Build a smaller but meaningful set of Crystal Reports.

Recommended:

Crystal Report 1

Tenant Payment History

Crystal Report 2

Property Rent Roll

Crystal Report 3

Property Income & Expense

The project doesn't need 14 Crystal Reports.

The objective is to show:

“I have actually worked with both reporting environments.”

This aligns with the job description's explicit mention of SSRS and Crystal Report Writer.

27. React Application

The frontend should remain professional but deliberately restrained.

Main navigation

Dashboard
Properties
Tenants
Leases
Payments
Collections
Maintenance
Vendors
Finance
Reports
Administration

Dashboard

Show:

Total Properties
Total Units
Occupancy %
Monthly Revenue
Outstanding Rent
Collection %
Open Maintenance
Expiring Leases

With charts for:

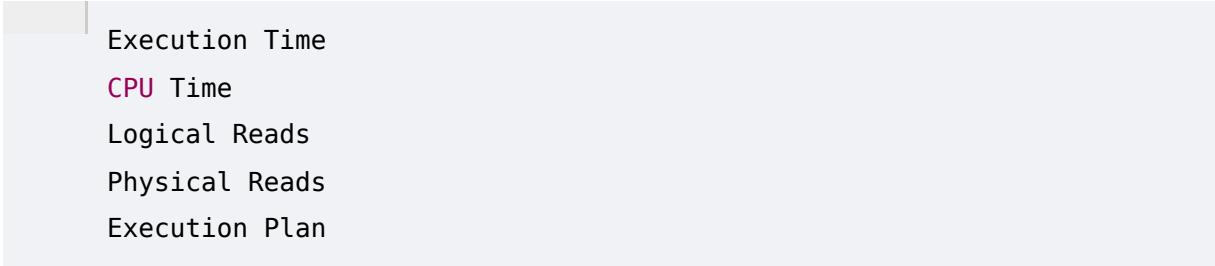
Occupancy trend
Revenue trend
Collections
Delinquency aging
Maintenance

The dashboard is primarily a **consumer of database/reporting capabilities**, not the centerpiece.

28. Performance Engineering

This is one of the highest-value parts of the project.

For selected complex queries, measure:



- Execution Time
- CPU Time
- Logical Reads
- Physical Reads
- Execution Plan

Then perform controlled optimization.

29. Performance Case Studies

Create at least **5 documented optimization experiments**.

Case Study 1

Slow property occupancy query

Case Study 2

Large payment-history query

Case Study 3

Monthly collection aggregation

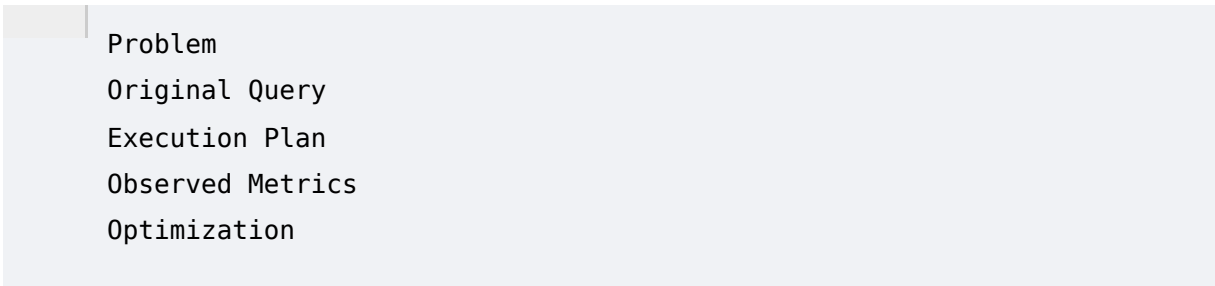
Case Study 4

Delinquency report

Case Study 5

Property financial summary

For each:



- Problem
- Original Query
- Execution Plan
- Observed Metrics
- Optimization


```
New Query
New Execution Plan
New Metrics
Analysis
```

30. Indexing Strategy

Demonstrate:

- Clustered indexes
- Non-clustered indexes
- Composite indexes
- Included columns where justified

Document:

```
Why this index exists
Which query benefits
Potential trade-offs
```

This is more impressive than simply saying "created indexes."

31. Important Rule About Performance Claims

Do **not** put fake numbers in the README or resume.

For example:

```
"Reduced execution time from 2.3 seconds to 40 ms"
```

should only appear if you actually measure it.

Instead:

```
"Analyzed execution plans and reduced query execution time through indexing
and query restructuring."
```

Then insert your actual benchmark.

This keeps the project technically credible.

32. Query Optimization Documentation

Create:

```
/docs/performance/
```

with files such as:

```
01-property-occupancy.md
02-payment-history.md
03-rent-collection.md
04-delinquency.md
05-financial-summary.md
```

Each should include before/after evidence.

33. Testing Strategy

Testing is explicitly relevant to the Yardi role because the posting mentions unit testing of software enhancements.

Unit Tests

Test:

- Late-fee calculation
 - Outstanding balance
 - Lease status
 - Rent generation
 - Payment validation
 - Delinquency classification
-

Integration Tests

Test complete workflows:

```
Create Lease
→ Generate Rent
→ Make Partial Payment
→ Calculate Balance
→ Apply Late Fee
→ Generate Delinquency
```

Database Tests

Test:

- Constraints
- Stored procedures

- Transactions
 - Trigger behavior
 - Referential integrity
-

Report Tests

Validate:

- Parameter filtering
 - Totals
 - Aggregations
 - Date ranges
 - Null handling
 - Export output
-

34. Error Handling

Application should gracefully handle:

```
Duplicate records
Invalid lease
Invalid payment
Negative amount
Payment exceeding valid balance
Missing tenant
Expired lease
Unauthorized access
Database failure
Report generation failure
```

The user should receive meaningful errors rather than raw database exceptions.

35. Security Requirements

Implement:

- Password hashing
- Authentication
- Role-based authorization
- API authorization
- Input validation

- Parameterized SQL / safe data access
- Audit logging
- Least-privilege principles

Sensitive tenant/payment data should not be exposed across roles.

36. SQL Folder Structure

The GitHub repository should make your SQL expertise immediately visible.

```
database/  
|  
├─ 01_schema/  
├─ 02_tables/  
├─ 03_constraints/  
├─ 04_seed_data/  
├─ 05_views/  
├─ 06_functions/  
├─ 07_stored_procedures/  
├─ 08_triggers/  
├─ 09_indexes/  
├─ 10_reports/  
├─ 11_test_scripts/  
└─ 12_performance/
```

This is much stronger than placing everything inside one `.sql` file.

37. Suggested Database Tables

A reasonable initial schema:

```
Users  
Roles  
UserRoles  
  
Owners  
Properties  
Buildings  
Units  
  
Tenants  
TenantContacts
```

```
Leases
LeaseTenants
LeaseRentHistory

RentCharges
```

The exact schema should be normalized appropriately rather than following this list blindly.

38. Key Stored Procedures

Recommended core procedures:

```
usp_CreateLease
usp_GenerateMonthlyRent
usp_RecordPayment
usp_RecordPartialPayment
usp_CalculateOutstandingBalance
usp_ApplyLateFees
usp_GenerateDelinquency
usp_RecordCollectionActivity
usp_ProcessLeaseRenewal
usp_CreateMaintenanceRequest
usp_AssignWorkOrder
usp_CloseMaintenanceRequest
```

Reporting procedures:

```
usp_GetOccupancyReport
usp_GetRentCollectionReport
usp_GetDelinquencyReport
usp_GetRentRoll
usp_GetLeaseExpiryReport
usp_GetTenantPaymentHistory
usp_GetPropertyFinancialSummary
```

39. Key Business Rules

Examples:

BR-01

An occupied unit must have at least one active lease.

BR-02

A lease cannot begin after its end date.

BR-03

A payment cannot be recorded against an invalid lease.

BR-04

A partial payment reduces outstanding balance but does not automatically mark a rent charge as fully settled.

BR-05

Late fees apply only after the configured grace period.

BR-06

Delinquency status depends on outstanding amount and overdue duration.

BR-07

A terminated lease cannot generate new rent charges.

BR-08

A closed maintenance request cannot receive further work without reopening.

BR-09

Only authorized roles can access financial data.

BR-10

Payment processing must be atomic.

40. API Design

Example endpoints:

```
POST /api/auth/login
```

```
GET /api/properties
```

```
POST /api/properties
```

```
GET /api/units
```

```
GET /api/tenants
```

```
POST /api/tenants
```

```
GET    /api/leases
POST   /api/leases

GET    /api/payments
POST   /api/payments

POST   /api/payments/partial
```

Reporting APIs can either return data to React or trigger/serve generated reports depending on implementation.

41. Reporting Workflow

Example:

```
User selects:
Property = Sunrise Residency
Month = August 2026
```

↓

React

↓

API

↓

Reporting Service

↓

Being able to explain this pipeline during the technical interview is a major objective.

42. Dashboard Analytics

The React dashboard should use SQL-derived KPIs.

KPI calculations

```
Occupancy Rate
Collection Rate
Outstanding Rent
Average Days to Collect
Lease Renewal Rate
Maintenance Resolution Rate
Net Property Income
```

Avoid calculating critical financial metrics independently in multiple frontend components.

There should be a controlled source of truth.

43. Support / Troubleshooting Simulation

Because the role involves SQL application support, add a small:

System Diagnostics

section.

Example:

```
Database Connectivity
Report Service Status
Last Successful Report
Failed Jobs
Slow Queries
Recent Errors
```

And a troubleshooting log:

```
Incident
Root Cause
Resolution
Preventive Action
```

This gives you something concrete to discuss when asked:

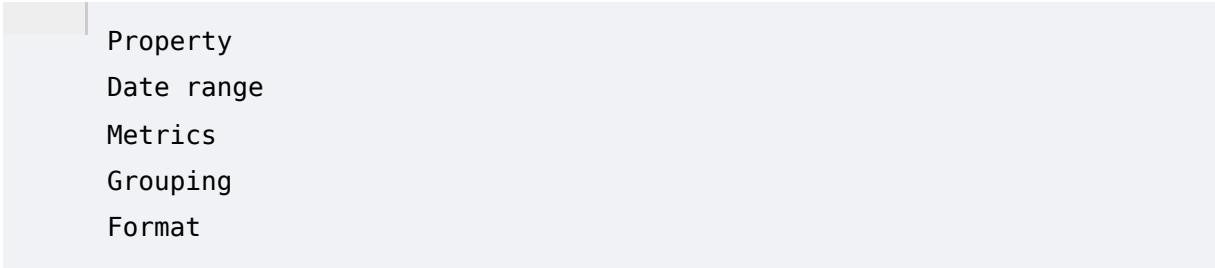
```
"How would you troubleshoot a production issue?"
```

44. Custom Solution Simulation

Create a feature called:

Client Report Builder

A property manager selects:



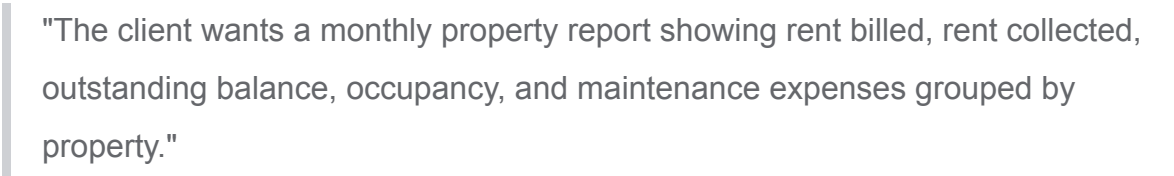
- Property
- Date range
- Metrics
- Grouping
- Format

and generates a customized report.

This directly demonstrates the concept of developing a **custom solution according to client requirements**, which appears in the Yardi role description.

45. Example Client Requirement

Document a simulated request:



"The client wants a monthly property report showing rent billed, rent collected, outstanding balance, occupancy, and maintenance expenses grouped by property."

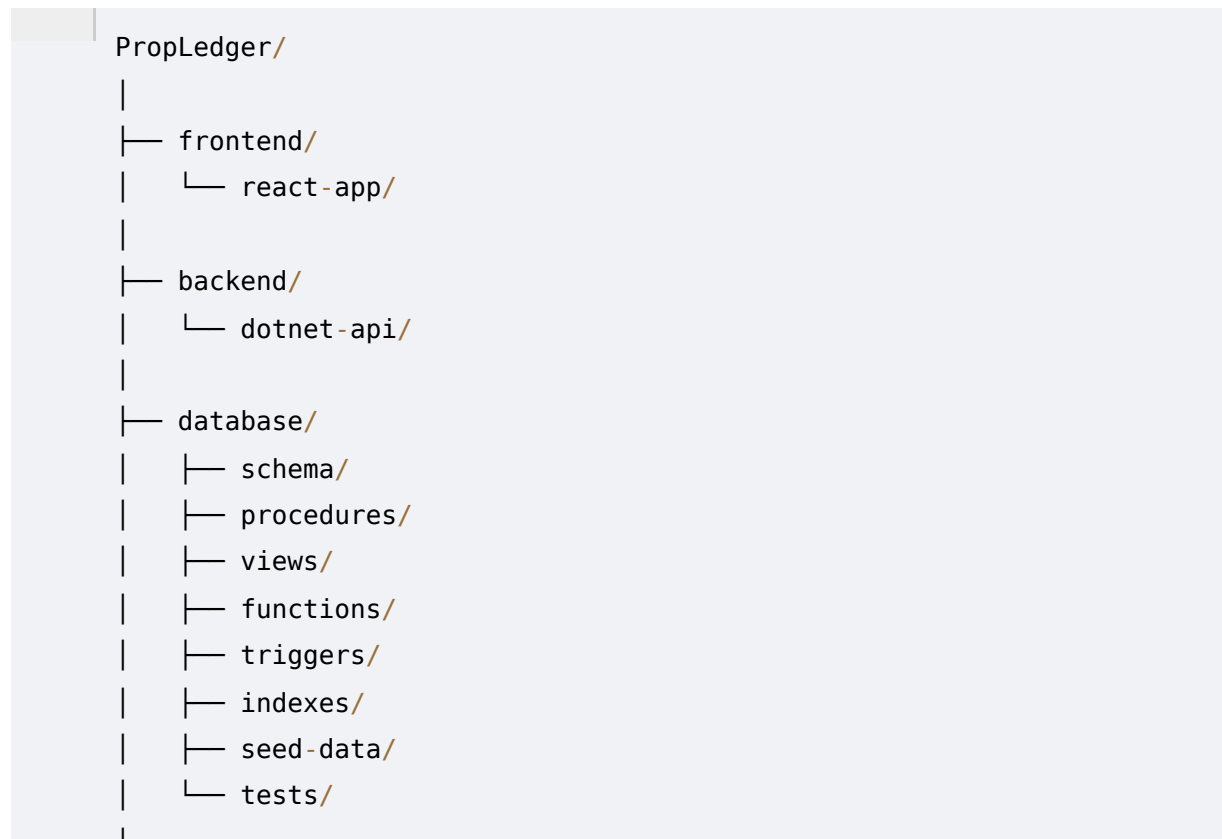
Your workflow:



```
graph TD; Requirement --> Analysis; Analysis --> DatabaseQuery[Database Query]; DatabaseQuery --> StoredProcedure[Stored Procedure]; StoredProcedure --> ReportDataset[Report Dataset]; ReportDataset --> SSRSReport[SSRS Report]; SSRSReport --> Testing; Testing --> Deployment
```

This is an excellent interview story.

46. Repository Structure



47. Development Phases

Phase 1 — Database Foundation

Build:

- ER model
- Normalized schema
- Constraints
- Seed data
- Core relationships

Deliverable: functional SQL Server database.

Phase 2 — Advanced SQL

Implement:

- Views
- CTEs
- Window functions

- PIVOT
- Functions
- Stored procedures
- Triggers
- Transactions

Deliverable: advanced SQL portfolio.

Phase 3 — Business Workflows

Implement:

```
Lease
→ Rent
→ Payment
→ Partial Payment
→ Late Fee
→ Delinquency
→ Collection
```

and:

```
Maintenance Request
→ Assignment
→ Resolution
→ Cost
```

Phase 4 — API

Implement ASP.NET Core API.

Phase 5 — React

Build the management interface.

Phase 6 — SSRS

Build the major operational reports.

Phase 7 — Crystal Reports

Build the selected Crystal Reports.

Phase 8 — Performance Engineering

Run:

```
Baseline
→ Analyze
→ Optimize
→ Re-measure
→ Document
```

Phase 9 — Testing

Implement:

```
Unit
Integration
SQL
Report
Performance
```

Phase 10 — Interview Packaging

Prepare:

- README
 - Architecture diagram
 - ER diagram
 - SQL examples
 - Query optimization case studies
 - Screenshots
 - Report samples
 - Testing evidence
 - Demo script
-

48. MVP

The MVP should contain:

```
Properties
Units
Tenants
```

```
Leases
Payments
Rent
Late Fees
Delinquency
Maintenance

Advanced SQL
Stored Procedures
Views
Transactions
Indexes

React UI
```

49. Phase 2 Features

After MVP:

```
Owners
Vendors
Expenses
Invoices
Collections
Security Deposits
Lease Renewals
Crystal Reports
Custom Report Builder
Diagnostics
Advanced Performance Lab
```

This prevents the project from becoming unmanageably large.

50. Success Criteria

The project is considered technically successful when you can demonstrate all of the following:

SQL

You can explain and demonstrate:

- complex joins

- CTEs
- recursive CTE
- window functions
- PIVOT
- stored procedures
- functions
- triggers
- transactions
- indexes

Reporting

You have functional:

- SSRS reports
- Crystal Reports
- parameterized reports
- PDF/Excel output

Performance

You can show:

```
Before execution plan
After execution plan
Before metrics
After metrics
Reason for optimization
```

Application

Users can perform real workflows from React.

Testing

You can demonstrate automated and database tests.

Interview

You can explain the architecture without relying on memorized terminology.

51. Resume Positioning

Once genuinely implemented, the project could eventually appear on your resume as:

PropLedger — Property Management & Analytics Platform

React.js | ASP.NET Core | SQL Server | SSRS | Crystal Reports

Possible bullets:

Designed and developed a property-management platform for leases, rent collection, delinquency, maintenance and property financials using SQL Server and React.

Implemented complex SQL solutions using CTEs, window functions, PIVOT, stored procedures, triggers and transactional workflows for operational and financial reporting.

Built SSRS and Crystal Reports for occupancy, rent collection, delinquency, lease expiry, rent roll and property financial analysis.

Performed execution-plan analysis, indexing and query optimization experiments to measure and improve database/report performance.

The **actual metrics should be inserted only after you benchmark the system.**

52. Interview Narrative

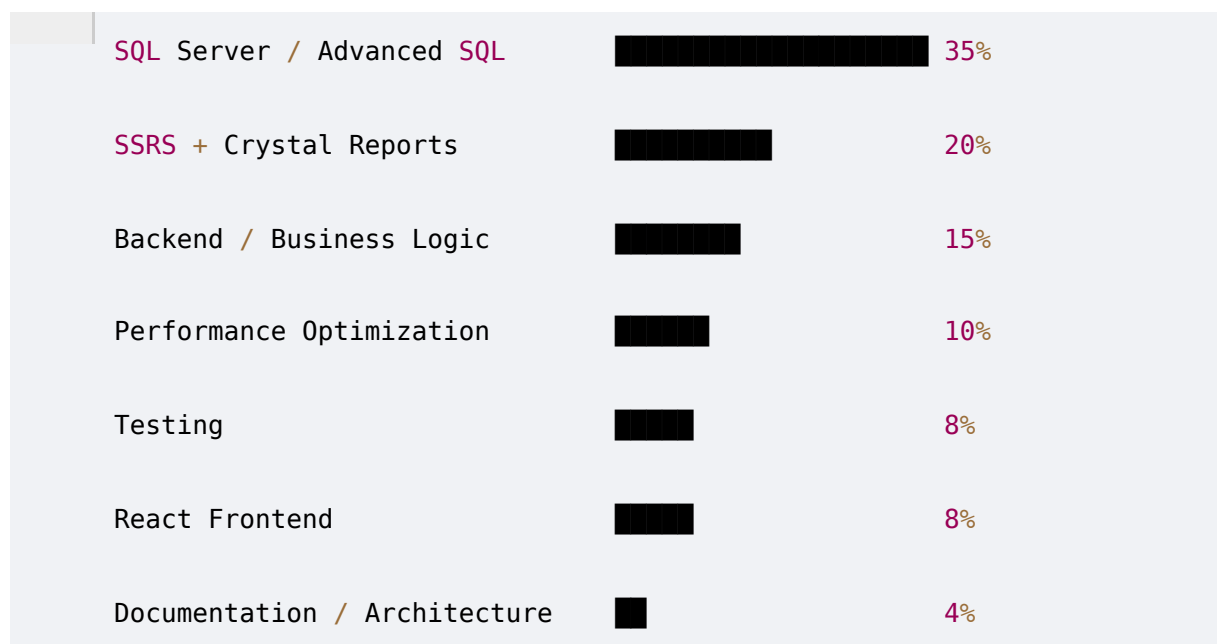
Your 60-second explanation should eventually sound approximately like this:

"I built PropLedger as a property-management and analytics platform using React, ASP.NET Core and SQL Server. Instead of treating it as a normal CRUD application, I focused heavily on database design, complex SQL, transactional payment processing and reporting. The system models properties, units, tenants, leases, rent, payments, delinquency and maintenance. I built SSRS and Crystal Reports for operational reporting, and I also performed execution-plan analysis and indexing experiments to optimize complex queries. I added automated tests for important business workflows such as partial payments, late fees and delinquency."

That is a **very strong project story for this particular role.**

53. Final Technical Priority

Given your choices, I would deliberately allocate the project effort like this:



That balance is intentional.

The Yardi notice describes the role around **SQL application support, custom solutions, reporting, complex SQL, optimization and testing**, so your project should make those capabilities impossible to miss.